Design Review

Submission Detail	Student Entry
GROUP ID	LAB-P1 SE06
GITHUB REPO LINK	https://github.com/adoreblvnk/whattheflip

Purpose

Document the system requirements, hardware connections, RTOS task structure, behavioural models, assumptions, and AI-supported design reasoning before major implementation. Treat this Week 6 submission as an architecture freeze before full integration and Tribunal 1. The design may change later, but major changes must be justified with engineering evidence.

How to use this template

- This is an architecture-freeze document. The team may improve the design later, but major changes must be justified during Tribunal 1.
- Use AI assistants for ideation, critique, and gap checking. Do not paste AI output directly as the final design.
- Summarise, challenge, correct, and justify AI suggestions using engineering reasoning.
- Keep all artefacts consistent: requirements, pin layout, RTOS task table, task interaction diagram, statecharts, assumptions, and AI evidence.

Team Role and Subsystem Ownership

Identify each buddy's primary subsystem responsibility. The role does not need to match the examples exactly, but every major hardware and software responsibility should have a clear owner.

Buddy	Student ID	Primary subsystem / responsibility
Tan Yan Qi	2500030	**Wired Protocol Auto-Discovery Engine:** 4-pin signal timing acquisition, rise-time measurement, UART/I2C classification, and dynamic pin role mapping
Annie Goh	2503469	**Physical Chaos & Fault Injection Engine:** Frame-synchronized glitch injection (bit-flips, I2C ACK suppression, clock stretching), bus contention protection, and hardware E-stop driver release.
Zayne Lee	2503844	**Wireless Multi-Protocol Sniffer Engine:** ESP32-C6 2.4 GHz promiscuous radio capture for Wi-Fi 6 (802.11ax) management frames and IEEE 802.15.4 (Zigbee 3.0), extracting RSSI, channel, and timestamps.
Ryan Ke	2501911	**Telemetry Ingestion & PCAP Storage Engine:** Zero-copy fixed memory ring buffers, packet serialization into standard libpcap format (.pcap), and high-speed FAT32 MicroSD card logging.
Joseph Poon	2501079	**System Supervisor, Web UI & Victim Control:** uT-Kernel supervisor coordination, WebSocket live streaming dashboard over local Wi-Fi, hardware E-stop interrupt handling, NeoPixel/buzzer indicators, and victim stimulus control.

Submission Instructions

- Complete every required section.
- Diagrams may be produced using any suitable tool, but labels must remain readable when inserted into this document.
- AI assistants may support drafting. The team remains responsible for technical correctness, consistency, and the ability to explain every decision.
- Use consistent names across the requirements, hardware sketch, task table, task interaction diagram, and statecharts.

1. Requirements

State what the system must do and the important quality or timing constraints. Requirements should be specific, testable, and traceable to the proposed design (you may add more FR and/or NFR as needed).

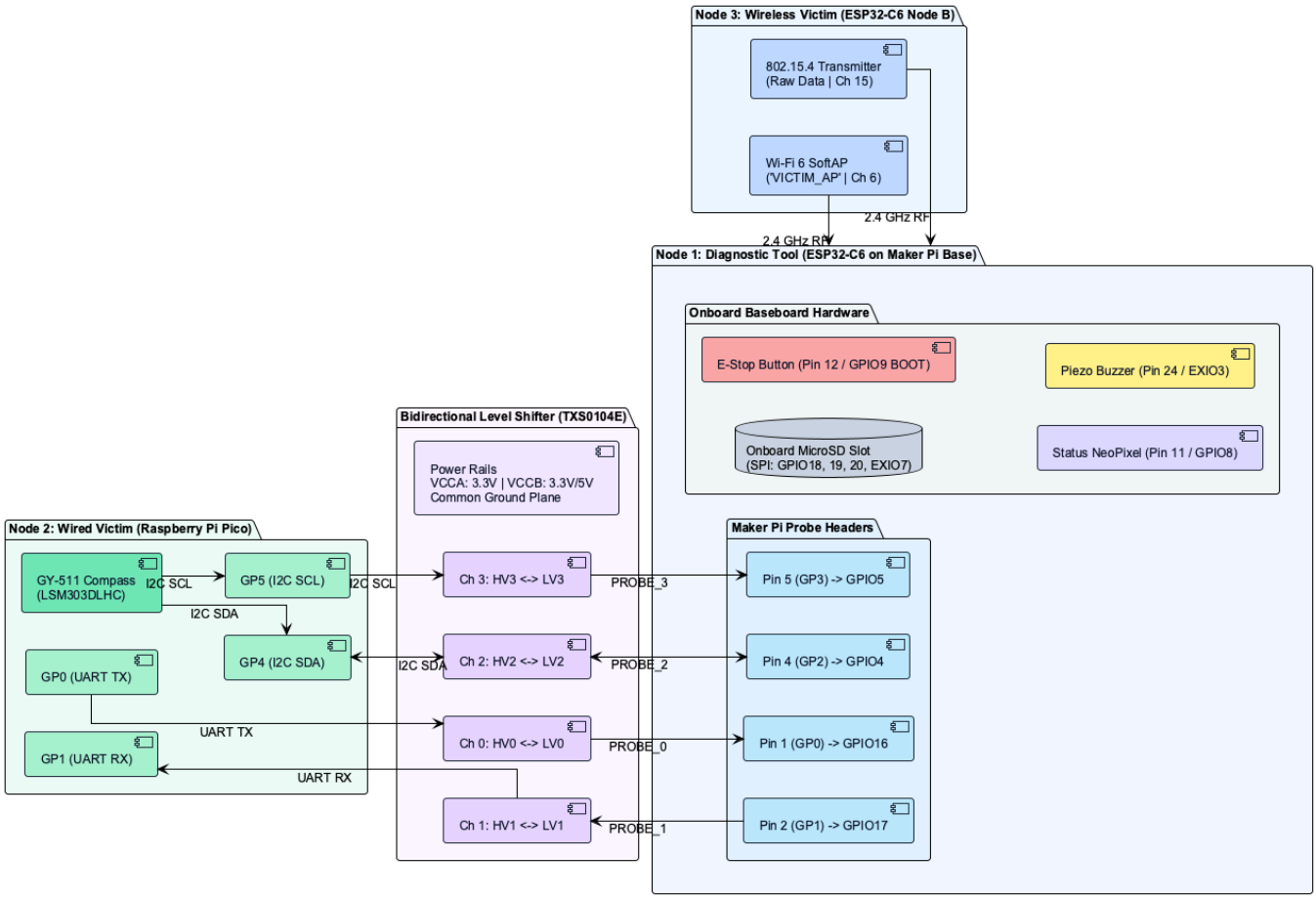
1.1 Functional Requirements

ID	Requirement Description
FR1	The system shall passively acquire digital signals across 4 probe pins.
FR2	The system shall automatically identify and classify active UART, I2C communication signals across the following operating limits: **UART:** Standard baud rates from 9600 to 115,200 baud (±2%). **I2C:** Standard clock frequencies of 100 kHz and 400 kHz (±2%).
FR3	The system shall remap any of the 4 probe pins to any internal protocol decoding channel without physical rewiring
FR4	The system shall capture 802.11 management frames (beacons, probe requests) and IEEE 802.15.4 frames across 2.4 GHz channels 11-26, extracting received signal strength (RSSI, ±3 dB), channel ID, and timestamps (±1μs).
FR5	When armed by the user, the system shall inject bounded protocol faults into the target bus
FR6	When armed by the user, the system shall transmit over-the-air stress frames (including 802.11 deauthentication frames) targeting the testbed wireless node
FR7	On actuation of the emergency stop input, the system shall disarm all active output drivers
FR8	The system shall record captured packets, each with a timestamp (±1μs) and raw packet bytes, to non-volatile removable media in a format importable by standard packet-analysis tooling.
FR9	The system shall broadcast decoded packet summaries and bus error statistics over a local wireless interface to connected client dashboards Upon detection of a system or bus fault condition, the system shall transmit a prioritized fault alert notification to connected client dashboards.

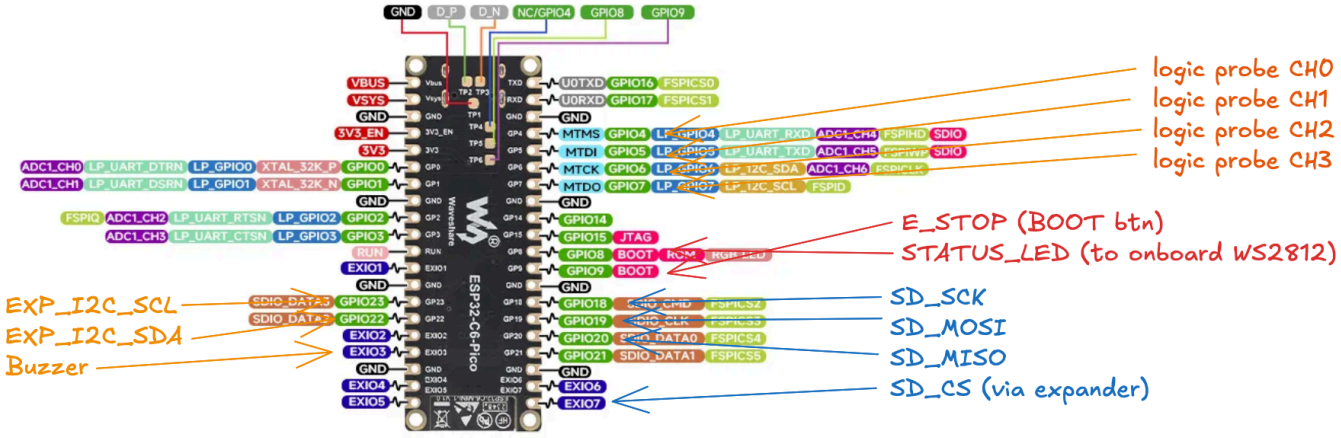
1.2 Non-Functional Requirements

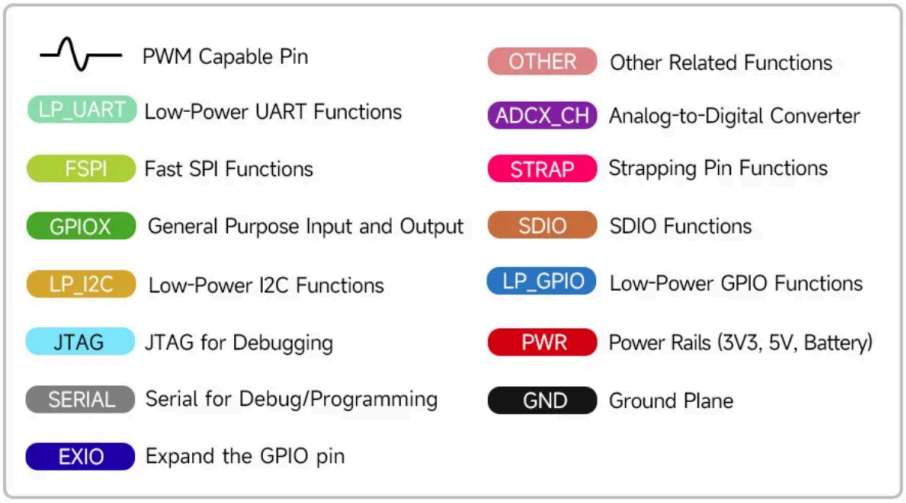
ID	Requirement Description
NFR1	The system shall acquire digital signals with a timing resolution of ≤ 1µs
NFR2	The probe interface shall tolerate continuous input voltages up to 3.3 V±5% without presenting more than 3.6 V to any microcontroller pad.
NFR3	The system shall output the protocol classification result within ≤ 10 ms of continuous bus activity.
NFR4	The system shall record packet arrival timestamps with an accuracy within ±1µs
NFR5	The firmware shall perform zero dynamic heap allocations during active monitoring and fault-injection phases (0 allocations measured over a ≥ 10 min continuous run).
NFR6	The system shall achieve a classification accuracy of ≥ 95% across all supported protocol configurations
NFR7	The system shall complete the pin remapping and return to a decode-ready state within ≤10 ms of receiving a remapping command.
NFR8	Fault-injection pulses shall synchronize to target bus trigger edges with a timing jitter of < 500 ns.
NFR9	The capture pipeline shall sustain a ≤ 0.1% packet drop rate for continuous wired streams up to 500 kbps, and shall buffer wireless bursts of ≥ 32 consecutive frames without loss.
NFR10	The system shall begin transmission of the target frames within ≤100 ms of receiving the arm command.
NFR11	The emergency stop and safety disarm path shall preempt all background processing within ≤ 5 ms of switch contact.

2. Hardware Integration Pin Layout



Hardware Interconnect & Maker Pi Base Layout





ESP32 C6 Pin Allocation

Device / Module	Signal Name	Sensor label	ESP32 C6 GPIO Pin	Interface	Notes / RTOS Task Owner
Wired-probe channel 0	PROBE_0	TX (Pico Victim A GP0)	GPIO4	GPIO / EDGE IRQ	Shared (AutoDiscovery / Fuzzer). Native GPIO. Direct 3.3V wire to Victim A GP0 (UART TX).
Wired-probe channel 1	PROBE_1	RX (Pico Victim B GP1)	GPIO5	GPIO / EDGE IRQ	Shared (AutoDiscovery / Fuzzer). Native GPIO. Direct 3.3V wire to Victim B GP1 (UART RX).
Wired-probe channel 2	PROBE_2	SDA (GY-511 / Victim A GP4)	GPIO6	GPIO / EDGE IRQ	Shared (AutoDiscovery / Fuzzer). Native GPIO. Direct 3.3V wire to Victim A GP4 / GY-511 SDA.
Wired-probe channel 3	PROBE_3	SCL (GY-511 / Victim A GP5)	GPIO7	GPIO / EDGE IRQ	Shared (AutoDiscovery / Fuzzer). Native GPIO. Direct 3.3V wire to Victim A GP5 / GY-511 SCL. (Note: Strapping pin, see A7).
Maker Pi MicroSD	SD_SCK	CLK (MicroSD Pin 5)	GP18	Hardware SPI	Storage_Task. 10 MHz SPI Clock (FSPI).

Device / Module	Signal Name	Sensor label	ESP32 C6 GPIO Pin	Interface	Notes / RTOS Task Owner
Maker Pi MicroSD	SD_MOSI	DI / CMD (MicroSD Pin 2)	GP19	Hardware SPI	Storage_Task.
Maker Pi MicroSD	SD_MISO	DO / DAT0 (MicroSD Pin 7)	GP20	Hardware SPI	Storage_Task.
IO Expander	SD_CS	CS / DAT3 (MicroSD Pin 1)	EXIO7	IO Expander	Storage_Task. Active-Low Chip Select driven via onboard TCA9554 expander pin EXIO7.
ESP32 C6 BOOT	E_STOP_BTN	GP9	GPIO9	GPIO Input	Safety_Supervisor_Task. Active-Low, $\leq 5ms$ High-Z interrupt (avoids I2C short).
ESP32 C6 WS2812	STATUS_LED	GPIO8	GPIO8	WS2812	Safety_Supervisor_Task. Onboard WS2812 Addressable RGB LED (Green: Ready, Blue: Sniff, Red: Fault).
Maker Pi Buzzer	BUZZER_OUT	PIEZO + (Onboard buzzer)	EXIO3	PWM / Audio	Safety_Supervisor_Task. Piezo buzzer (header position GP18), driven via onboard TCA9554 expander (EXIO3).
TCA	EXP_I2C	SCL / SDA (TCA9554 PWR)	GPIO22 / 23	I2C Master Bus	System Init. Drives the ESP32-C6-Pico onboard TCA9554PWR expander (EXIO1–EXIO7). Baseboard Buttons 1 & 2 must NOT be pressed.

3. RTOS Task Table

List the ALL the proposed RTOS tasks. Priorities are preliminary at Week 6 and must be justified.

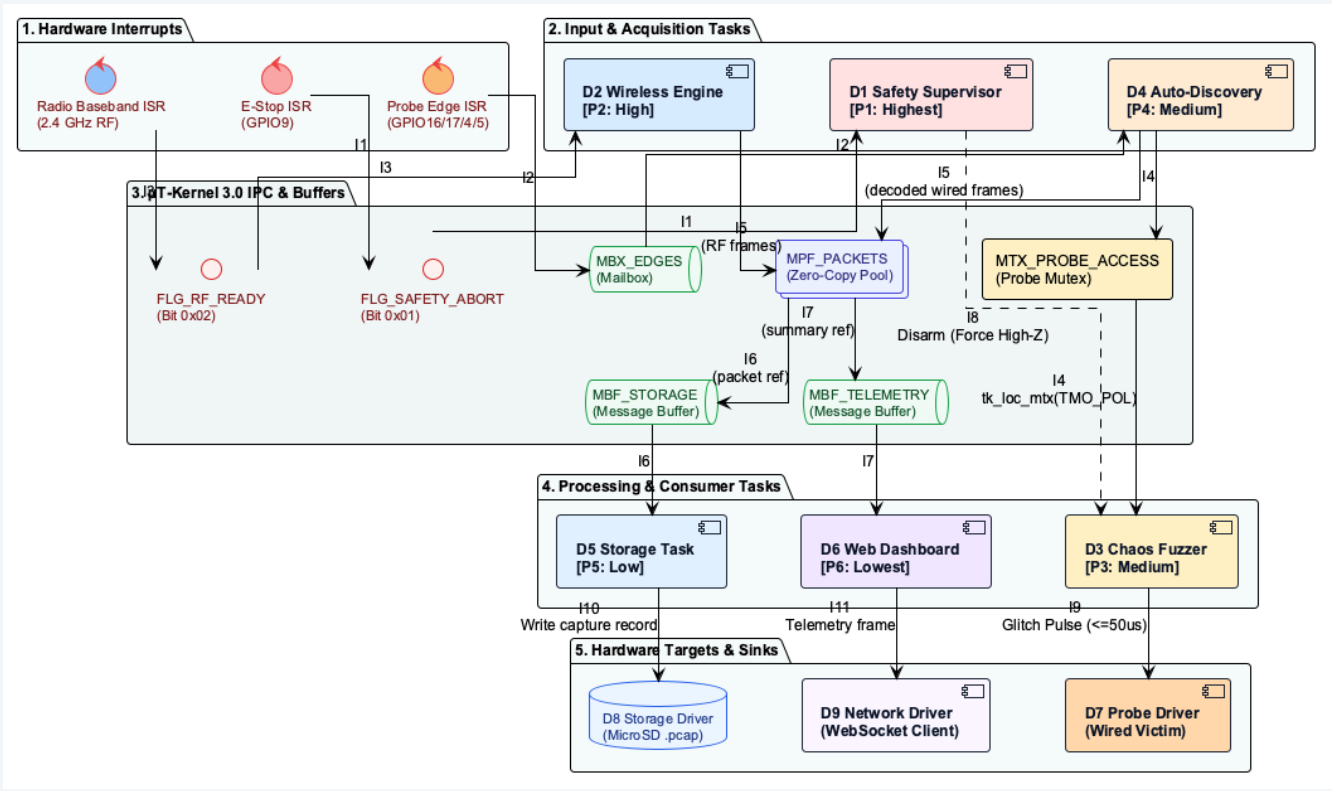
Task	Purpose	Inputs	Outputs	Trigger	Priority
`Safety_Supervisor_Task`	Monitor emergency stop, system faults, and enforce High-Z probe isolation	GPIO9 E-stop IRQ, watchdog, bus short flags	High-Z disarm request to probe driver, RGB LED, buzzer	Event-driven (IRQ / Fault flag)	**Highest (1)**
`Wireless_Engine_Task`	Drain 2.4 GHz radio baseband FIFOs and extract packet metadata	Radio promiscuous RX baseband event	Ingest packet records into memory pool	Event-driven (Radio Baseband IRQ)	**High (2)**
`Chaos_Fuzzer_Task`	Inject synchronized glitch pulses into target bus transmissions	Trigger pattern config, edge events	Timed glitch pulse request to probe driver (bit-flip / NACK)	Event-driven (Frame sync event)	**Medium (3)**
`AutoDiscovery_Task`	Sample edge transitions, measure pulse widths, and resolve pin mappings	Probe edge FIFO timestamps	Resolved protocol ID, pin role table	Periodic (50 ms) / Edge event	**Medium (4)**
`Storage_Task`	Serialize packet buffers to capture files on removable media	Ingested packet records from message buffer	Disk sector writes via storage driver	Event-driven (Buffer threshold)	**Low (5)**
`Web_Dashboard_Task`	Serve dashboard assets and stream live telemetry over the wireless interface	Decoded packet headers, bus error stats	Telemetry frames to network driver	Periodic (500–1000 ms)	**Lowest (6)**

Priority and ownership justification:

`Safety_Supervisor_Task` is allocated Priority 1 (highest) to guarantee that an emergency stop actuation or electrical contention event preempts all other tasks within ≤ 5 ms, immediately floating all probe pins to protect connected target hardware. `Wireless_Engine_Task` is set to Priority 2 to prevent hardware radio FIFO overruns during dense 802.11 beacon bursts. `Chaos_Fuzzer_Task` sits at Priority 3 to ensure deterministic, sub-microsecond glitch timing when a target frame trigger is recognized. `AutoDiscovery_Task` sits at Priority 4 to perform edge timing analysis without preempting active frame capture. `Storage_Task` and `Web_Dashboard_Task` are assigned the lowest priorities (5 and 6) because disk and network operations are decoupled through pre-allocated fixed memory pools (MPF_PACKETS), ensuring slow SD card sector writes never block time-critical bus sampling.

4. Task Interaction Diagram

Show RTOS tasks, queues, semaphores, mutexes, event groups, notifications, and data flow. Clearly distinguish software tasks from hardware devices and interrupt service routines.

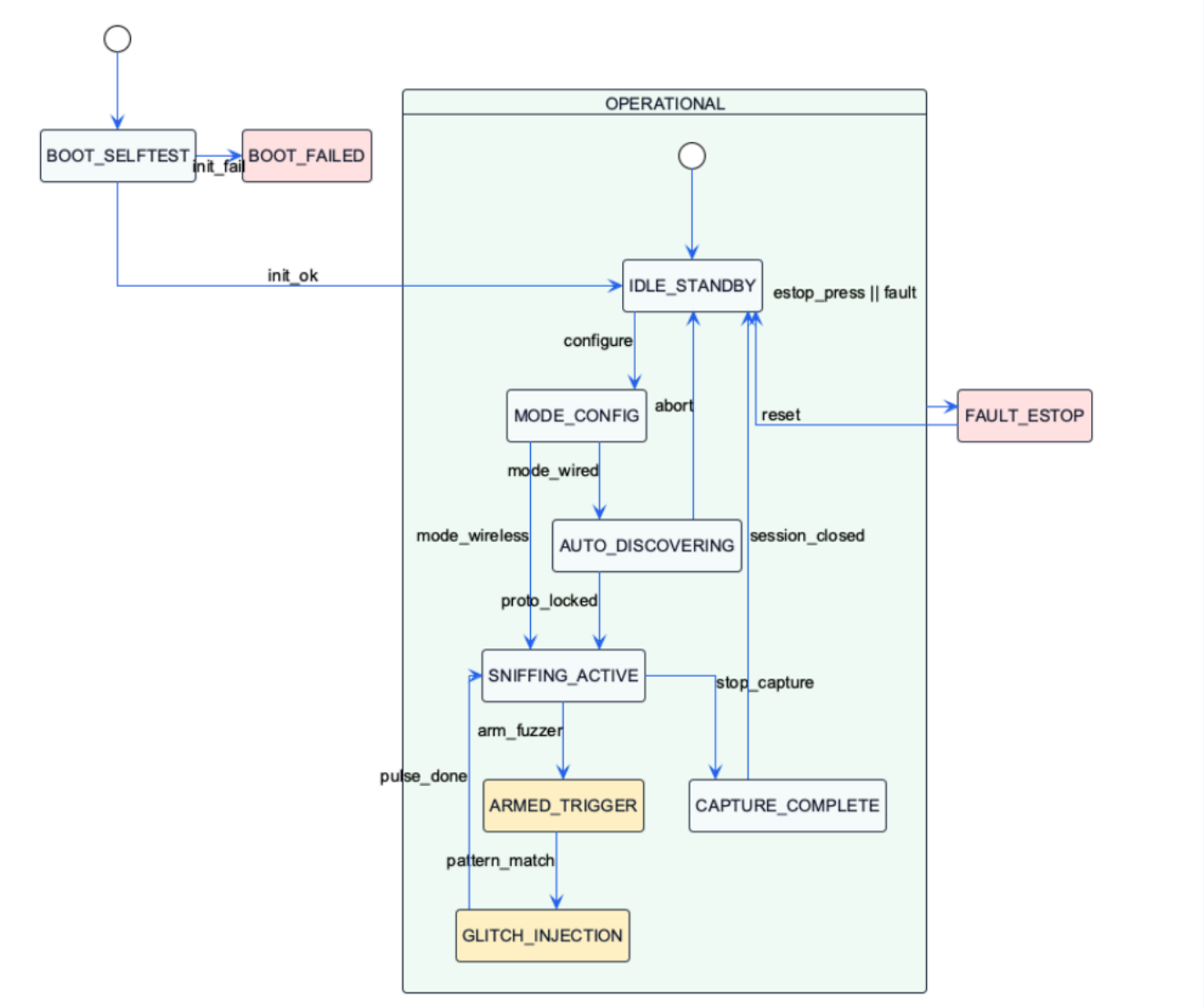


Brief explanation of the main communication paths:

- Safety Interrupt Path: When GPIO9 falls LOW, E-Stop ISR sets EventFlag FLG_SAFETY_ABORT , waking TSK_SAFETY (Priority 1) to directly write to GPIO output-enable registers, floating all probe pins to High-Z within $\leq 5ms$.
- Probe Arbitration & Timing Path: Probe Edge ISR captures hardware timer timestamps and enqueues them via Mailbox MBX_EDGES to TSK_DISCOVERY . Probe pin access between discovery and fuzzing is serialized via Mutex MTX_PROBE_ACCESS .
- Zero-Copy Ingestion & Storage Pipeline: When Radio ISR receives an 802.11 or 802.15.4 frame, it allocates a buffer from Fixed Memory Pool MPF_PACKETS without dynamic malloc . The populated packet is referenced into Message Buffers MBF_STORAGE and MBF_TELEMETRY . Even if a MicroSD card write incurs a $30ms$ latency spike, the pre-allocated memory pool absorbs incoming bursts without dropping frames.

5. System-Level Statechart

Model the operating states of the entire system. Label transition events or conditions. Include relevant fault, stop, or recovery behaviour.



Explain the main states and any important assumptions:

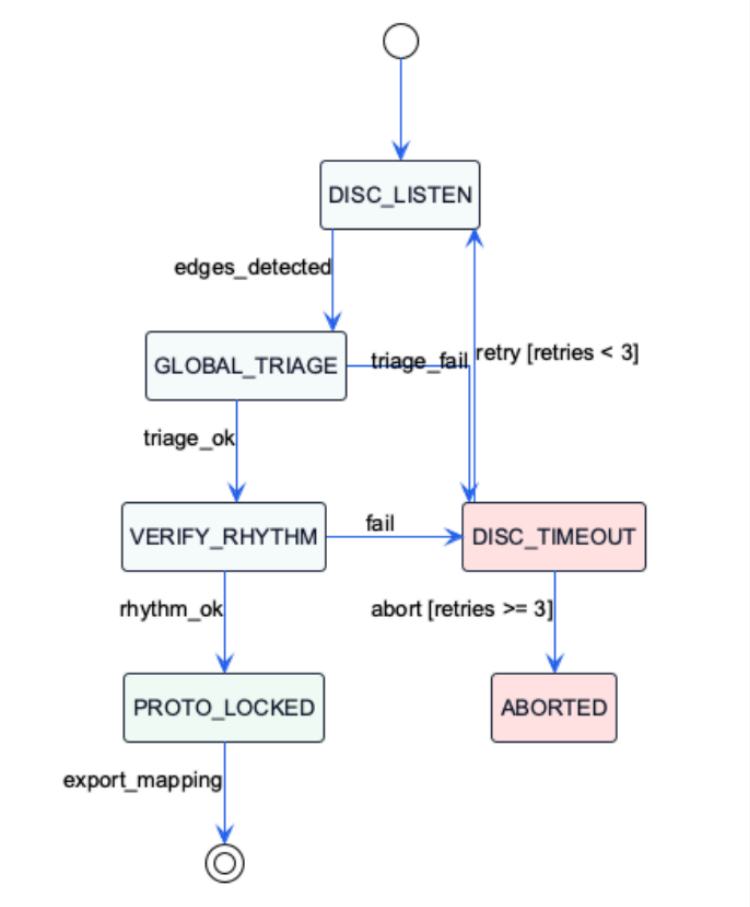
- BOOT_SELFTEST : Verifies MicroSD card mounting (storage SPI interface), and radio baseband initialization. Transitions to BOOT_FAILED if hardware initialization fails.
- BOOT_FAILED : Terminal fault state halting system execution with a blinking red LED if critical boot hardware is unreadable.
- IDLE_STANDBY : Default safe operational state. Probe pins are configured as High-Z inputs; fuzzer output drivers are completely disabled.
- MODE_CONFIG : Processes user configuration commands to set operating profiles (Wired vs. Wireless, RF channels, or baud search parameters).

- AUTO_DISCOVERING : Measures edge counts, pulse widths, and clock period stability across the 4 probes to resolve protocol type and pinout.
- SNIFFING_ACTIVE : Ingests packet streams into zero-copy memory pools, writing .pcap files to MicroSD and broadcasting live WebSocket telemetry.
- ARMED_TRIGGER : Fuzzer armed; synchronizes to target bus framing and awaits a matched address/header trigger pattern.
- GLITCH_INJECTION : Injects a precision in-flight glitch pulse (bit-flip or ACK suppression bounded to <50us) and verifies bus release.
- CAPTURE_COMPLETE : Flushes remaining packet ring buffers to MicroSD and closes active PCAP sessions before returning to IDLE_STANDBY .
- FAULT_ESTOP : Global emergency preemption state invoked by physical E-stop press or bus contention. Forces all probe lines to High-Z in $\leq 5ms$, sounds buzzer, and latches until manually reset.

6. Subsystem Statecharts

6.1 Subsystem Statechart 1

Field	Student Entry
Subsystem name	Wired Auto-Discovery Engine
Related RTOS task(s)	AutoDiscovery_Task (TSK_DISCOVERY)
Purpose	Passively classify unknown bus protocols and resolve signal pin assignments.
Reason a statechart is needed	Enforces a multi-stage sequential filter (pin triage → rhythm verification → confirmation) to reject noise spikes and prevent false protocol locks.

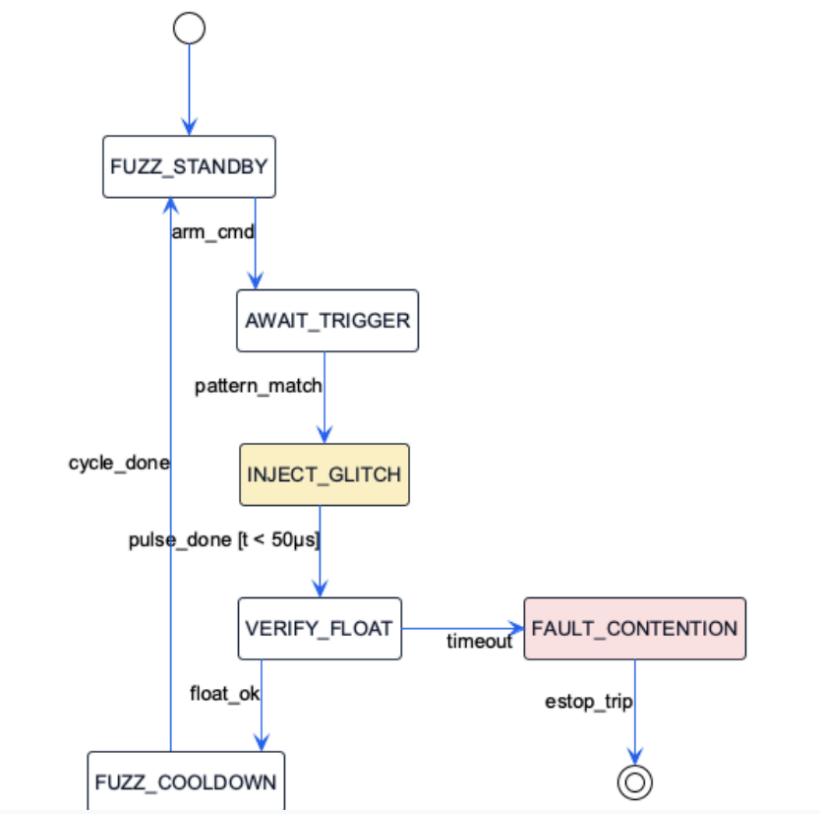


Brief explanation of states, events, and fault handling:

- DISC_LISTEN : Samples probe transitions. Upon receiving edge interrupts, moves to GLOBAL_TRIAGE .
- GLOBAL_TRIAGE : Applies exclusivity rules (1 pin = UART candidate, 2 pins = I2C candidate). If active pin count is invalid, triggers ev_triage_fail .
- VERIFY_RHYTHM : Evaluates baud rate pulse widths ($\pm$ 5% tolerance) or I2C clock period stability ($\pm$ 25% jitter bound).
- PROTO_LOCKED : Emits ev_proto_locked , updates shared pinout tables, and releases MTX_PROBE_ACCESS .
- Fault Handling: If rhythm verification fails or no valid protocol locks within $2s$, transitions to DISC_TIMEOUT , notifies the dashboard, and retries up to 3 times before resetting to DISC_LISTEN .

6.2 Subsystem Statechart 2

Field	Student Entry
Subsystem name	Physical Chaos & Fault Injection Engine
Related RTOS task(s)	Chaos_Fuzzer_Task (TSK_FUZZER)
Purpose	Inject synchronized non-destructive faults (bit-flips, ACK suppression, clock stretching) into active bus traffic.
Reason a statechart is needed	Guarantees that active electrical drive is strictly bounded in time, preventing bus shorts or damaged target drivers.



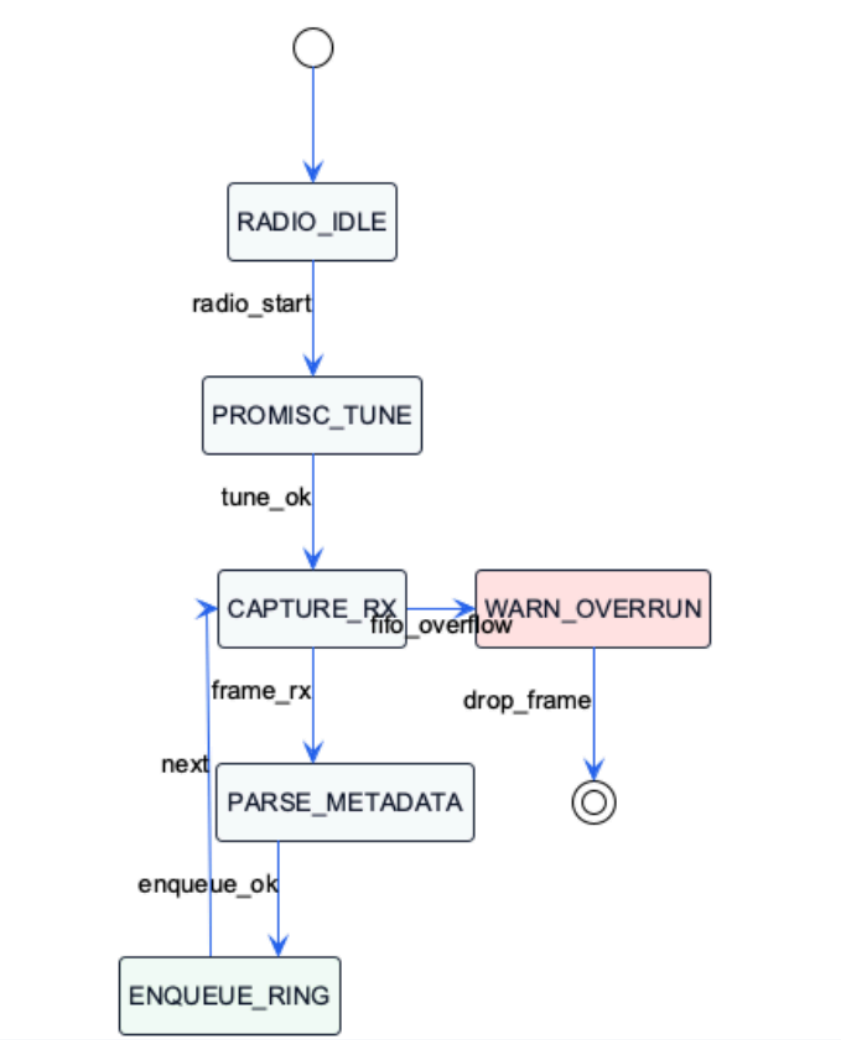
Brief explanation of states, events, and fault handling:

- FUZZ_STANDBY : Probes remain in High-Z mode until user issues arm command.
- AWAIT_TRIGGER : Evaluates incoming bytes to synchronize with the target frame header.
- INJECT_GLITCH : Drives the probe line for a single bit duration (e.g. 8.68us at 115200 baud).
- VERIFY_FLOAT : Re-samples line state to ensure the line returned to idle HIGH.
- Fault Handling: If a line remains driven or pulled LOW for >100us, FAULT_CONTENTION is asserted, immediately switching all outputs to High-Z and alerting

Safety_Supervisor_Task .

6.3 Subsystem Statechart 3

Field	Student Entry
Subsystem name	Wireless Multi-Protocol Sniffer
Related RTOS task(s)	Wireless_Engine_Task (TSK_WIRELESS)
Purpose	Intercept raw 802.11 and 802.15.4 packets and extract RF physical metadata.
Reason a statechart is needed	Manages radio baseband transitions and provides graceful degradation during high-throughput packet bursts.



Brief explanation of states, events, and fault handling:

- RADIO_IDLE : Radio disabled; power consumption minimized.
- PROMISC_TUNE : Configures channel filter and engages promiscuous mode.
- CAPTURE_RX : Drains hardware baseband FIFO upon packet arrival.
- PARSE_METADATA : Extracts RSSI, channel ID, and microsecond timestamps into standard PCAP frame format.
- Fault Handling: If the fixed memory pool reaches $> 85\%$ capacity, non-critical broadcast data frames are dropped while management/control frames are preserved, logging an overrun warning to telemetry.

7. Assumptions

List assumptions that the design depends on. Each assumption must state the risk if it is wrong and how the team will validate it before full integration. Have **at least** 1 assumption per buddy.

ID	Assumption	Risk if wrong	Validation method	Owner	Status
A1	Probe inputs resolve clean CMOS logic thresholds ($V_{IL} \leq 0.8V$, $V_{IH} \geq 2.0V$) without excessive ringing on 15 cm jumper lines.	Edge noise triggers spurious interrupts, causing false baud rate detection.	Capture 100 kHz I^2C transitions on an oscilloscope with 15 cm leads; verify rise time $t_r \leq 300ns$ and no false trigger interrupts.	Tan Yan Qi	Open
A2	ESP32-C6 GPIO pins can switch direction from High-Z Input to Driven Output in <1µs for targeted bit glitching.	Delayed output drive misses the target clock pulse, failing to inject the fault.	Program a GPIO toggle triggered by an external pulse; measure drive delay on an oscilloscope with <50 ns resolution.	Annie Goh	Open
A3	The ESP32-C6 promiscuous packet filter captures raw 802.11 management frames without dropping frames at burst rates up to 50–100 pkts/s.	Dropped beacon frames cause incomplete Wireshark trace exports.	Blast the sniffer with a 100 pkt/s beacon generator (Node 3); compare transmitted vs. logged frame sequence counters.	Zayne Lee	Open
A4	SPI FAT32 MicroSD write latency spikes (≤40 ms) can be fully absorbed by an 8 KB pre-allocated packet ring buffer.	Buffer overflow during SD write stalls drops incoming physical/wireless packets.	Simulate a 50 ms SD write stall while streaming UART traffic at 115200 baud; verify zero dropped bytes in memory logs.	Ryan Ke	Open
A5	The WebSocket telemetry server can stream 1–2 Hz packet updates over Wi-Fi without starving the µT-Kernel scheduler.	Network stack CPU load introduces jitter into physical bus sampling.	Benchmark system timer jitter while serving 3 concurrent WebSocket clients running telemetry streams at 1 Hz.	Joseph Poon	Open
A6	`GPIO15` (`PROBE_3` / Maker Pi `GP7`) functions as a JTAG source select strapping pin during ESP32-C6 boot; holding probes at High-Z or keeping victims quiescent during reset preserves default USB Serial/JTAG operation.	Victim line pulling `GPIO15` LOW during ESP32-C6 power-on redirects JTAG to external pins, requiring a board power cycle to restore USB JTAG debugging.	Connect victim I2C SCL to `GP7`; verify via oscilloscope and serial log; insert 1 kΩ series isolation resistor if boot contention is observed.	Tan Yan Qi / Annie	Open

8. AI-Supported Design Evidence

Document how AI was used to support the design. The aim is not to show that AI produced perfect answers, but to show that the team used AI critically, corrected weak suggestions, and made final engineering decisions.

8.1 Pre-Submission AI Ideation

Buddy	AI prompt or discussion used	Useful AI ideas	Weak or unrealistic AI ideas	Team decision
Yan Qi	Prompted AI on multi-pin signal triage heuristics for mixed UART and I2C lines.	AI proposed the "Global Pin Exclusivity Filter" (counting active lines before calculating timings).	AI suggested using a neural network classifier on the Pico to identify protocols.	Adopted the deterministic pulse-width heuristic; rejected neural networks as an unnecessary compute and memory burden.
Annie	Asked AI how to safely inject in-flight faults on open-drain I2C buses without frying master drivers.	AI suggested driving lines strictly during slave ACK clock pulses and adding an immediate High-Z release timeout.	AI suggested cutting the physical VCC line with a MOSFET to simulate device failure.	Implemented software-timed ACK suppression and physical E-stop High-Z override; rejected cutting VCC to prevent back-powering chips.
Zayne	Inquired whether ESP32-C6 can sniff connected BLE data channels promiscuously.	AI identified that ESP-IDF only supports BLE GAP advertisement scanning, not raw data channel sniffing.	AI initially claimed BLE promiscuous mode existed via standard APIs.	Critiqued and dropped Bluetooth link-layer sniffing; scoped wireless strictly to native Wi-Fi 6 + 802.15.4 promiscuous capture.
Ryan	Asked AI how to log high-rate packet streams to MicroSD without missing real-time deadlines.	AI suggested using pre-allocated fixed memory pools (MPF) and message buffers to decouple capture from SD block writes.	AI suggested writing raw text JSON strings to the SD card for each captured packet.	Adopted zero-copy fixed memory pools; rejected JSON in favor of compact binary libpcap format.
Joseph	Asked AI to evaluate whether a 0.96" OLED display should be included alongside the Wi-Fi dashboard.	AI highlighted that the OLED consumes 1 KB RAM, bus bandwidth, and cannot render complex hex packet trees.	AI suggested keeping both OLED and Web dashboard for redundancy.	Completely dropped the OLED screen; allocated resources to the WebSocket web dashboard and status NeoPixel.

8.2 Team-Reviewed Architecture Freeze

AI or initial idea	Team decision	Engineering reason	Affected artefact
Use a closed-loop testbed with 2 × RP2040 wired victims on a breadboard + 1 × ESP32-C6 wireless victim.	Accepted	Proves technical scope with dual dedicated targets on a shared breadboard: Victim A drives UART TX & I2C Master reading GY-511 sensor; Victim B acts as UART RX listener & I2C Slave. Enables simultaneous multi-protocol sniffing and chaos fuzzing without physical rewiring between demos.	Testbed topology, System overview, Hardware pinout
Mount ESP32-C6 on Cytron Maker Pi Pico Base carrier board.	Accepted	Eliminates external MicroSD breakout module by utilizing onboard SD slot; provides per-pin blue LED diagnostics and clean benchtop mounting.	Hardware pinout table, Schematic, Bill of Materials
Drop 0.96" OLED display in favor of Wi-Fi WebSocket dashboard.	Accepted	Eliminates I2C bus contention, saves 1 KB RAM, and provides far superior visual clarity during zoom tribunals.	Bill of Materials, Pinout table, Task table
Drop passive SPI sniffing from auto-discovery and fuzzing scope.	Accepted	Passive 4-wire SPI (SCK, MOSI, MISO, CS) consumes all 4 probe channels with zero remaining lines for concurrent buses. SPI lacks addressing/ACK framing for meaningful protocol fuzzing.	Protocol matrix, Task architecture, Statechart 1, R-1.2

8.3 Final AI-Vetted Gap Check

After completing the design, ask AI to act as a strict embedded systems reviewer. The review should identify missing requirements, unclear interfaces, untested assumptions, weak mitigations, timing concerns, task conflicts, or resource contention.

Use one of these Team Response options: **Fixed**, **Partially fixed**, **Accepted as risk**, **Deferred**, **Rejected with reason**, or **Needs validation**. The action and evidence columns must justify the selected response. **AI reviewer prompt used**	"Act as a strict embedded systems reviewer for whattheflip. Check requirements, pinout, RTOS task table, IPC architecture, statecharts, assumptions, and consistency. Identify missing requirements, unclear interfaces, untested assumptions, timing risks, task conflicts, and shared-resource problems."

AI-identified gap	Team response	Action taken	Evidence or reason
Maker Pi Base user buttons share header positions near the C6's committed I2C pins (GPIO22/GPIO23 to the onboard TCA9554 expander).	Fixed	Mandated the native BOOT switch on Pin 12 (GPIO9) for Emergency Stop, and left all three Maker Pi base user buttons unused.	Hardware integration Section 2.2 (pin table + safety invariant note).
Unclear probe line ownership between Auto-Discovery and Fuzzing tasks.	Fixed	Defined dynamic probe sharing arbitrated via Mutex MTX_PROBE_ACCESS.	Task table Section 3 and IPC diagram Section 4 updated.
MicroSD card write latency spikes (40 ms) may cause buffer overrun during burst captures.	Needs Validation	Sized fixed memory pool (MPF_PACKETS) to 8 KB to buffer up to 32 consecutive frames; validated via Assumption A4.	Assumption A4 documented with simulated 50 ms stall benchmark.
Single 2.4 GHz RF front-end cannot capture Wi-Fi and 802.15.4 simultaneously.	Accepted as Risk	Documented time-multiplexed capture constraint; system operates in distinct Wi-Fi or Zigbee listening modes.	Documented in Section 2 and Subsystem Statechart 3.

9. Design Consistency

Use this section to demonstrate that the design artefacts describe one coherent system rather than independent diagrams.

Check	Student Response
Which task owns each hardware device?	- AutoDiscovery_Task and Chaos_Fuzzer_Task dynamically share the 4 target probe lines on GP4–GP7 (ESP32-C6 GPIO4, GPIO5, GPIO6, GPIO7), arbitrated via Mutex MTX_PROBE_ACCESS. - Wireless_Engine_Task owns the 2.4 GHz radio transceiver baseband. - Storage_Task owns the Maker Pi Base onboard MicroSD card slot hardwired to header positions GP10, GP11, GP12, and GP15 (physical pins 14-16 and 20: ESP32-C6 GPIO18, GPIO19, GPIO20, and EXIO7). - Safety_Supervisor_Task owns the emergency stop button on Maker Pi Pin 12 (GPIO9 BOOT switch), status NeoPixel (ESP32-C6 onboard GPIO8), and the Maker Pi Base alert buzzer (EXIO3). - Web_Dashboard_Task owns the Wi-Fi AP network socket interface.
Which task has the highest priority, and why?	Safety_Supervisor_Task has the highest priority (Priority 1) because target hardware safety is the primary system invariant. If an electrical short occurs or the user presses the emergency stop, the system must immediately preempt all other tasks and drop probe pins to High-Z in ≤ 5 ms.
Which subsystem statechart corresponds to which RTOS task or subsystem?	- Subsystem Statechart 1 corresponds to AutoDiscovery_Task (TSK_DISCOVERY). - Subsystem Statechart 2 corresponds to Chaos_Fuzzer_Task (TSK_FUZZER). - Subsystem Statechart 3 corresponds to Wireless_Engine_Task (TSK_WIRELESS).
How are functional requirements reflected in the task and statechart design?	- FR1–FR3 (Probing, Auto-Discovery, Pin Routing) map to AutoDiscovery_Task and the AUTO_DISCOVERY state. - FR4 (Wireless Capture) maps to Wireless_Engine_Task and the PASSIVE_SNIFF state. - FR5–FR6 (Chaos Fault Injection) map to Chaos_Fuzzer_Task and the CHAOS_FUZZING state. - FR7 (Emergency Stop) maps to Safety_Supervisor_Task and the FAULT_ESTOP state. - FR8–FR9 (PCAP Storage and Web Streaming) map to Storage_Task and Web_Dashboard_Task.
What changes are still expected after the Week 6 review?	- Confirmation of empirical rise-time tolerances during Week 8 hardware wiring tests. - Fine-tuning of SPI DMA block transfer sizes and ring buffer depths based on measured SD write latency benchmarks. - Validation of assumptions before Week 10 Subsystems Check-in.

10. Appendix: Traceability Matrix

Requirement ID	Use Case / Sequence ID	LLD Function Name	Test ID	Expected Result
FR1 (Passive Probing)	UC-1.1 (Sample Bus Lines)	`probe_sample_edges()`	**TC-1.1**	Edge timestamps resolved to ≤ 1 µs; leakage ≤ 1 µA
FR2 (Auto-Discovery)	UC-1.2 (Classify Protocol)	`analyze_for_i2c()`, `analyze_for_uart()`	**TC-1.2**	Correct protocol identified within ≤ 500 ms for UART and I2C buses
FR3 (Dynamic Reconfig)	UC-1.3 (Map Peripheral Pins)	`gpio_matrix_route_pins()`	**TC-1.3**	Any probe pin routed to any decode channel; ready ≤ 10 ms, no rewiring
FR4 (Wi-Fi Capture)	UC-2.1 (Capture 802.11 Frames)	`wifi_promiscuous_rx_cb()`	**TC-2.1**	≥ 99% of injected management frames logged with RSSI/channel/timestamp
FR4 (Zigbee Capture)	UC-2.2 (Capture 802.15.4)	`ieee802154_rx_cb()`	**TC-2.2**	≥ 99% of injected 802.15.4 frames logged on channels 11 to 26
FR5 (I2C Fault Injection)	UC-3.1 (Suppress I2C ACK)	`fuzzer_inject_i2c_nack()`	**TC-3.1**	Target ACK converted to NACK; line released ≤ 50 µs
FR5 (UART Bit Flip)	UC-3.2 (Invert UART Bit)	`fuzzer_invert_uart_bit()`	**TC-3.2**	Target bit inverted in-flight; no residual bus drive
FR6 (Wireless Stress)	UC-3.3 (Inject Deauth Frame)	`wifi_80211_tx_deauth()`	**TC-3.3**	Deauth emitted ≤ 100 ms after arm; victim re-associates (see A6)
FR7 (Emergency Stop)	UC-4.1 (E-Stop Abort)	`safety_abort_to_high_z()`	**TC-4.1**	All probe lines High-Z ≤ 5 ms after switch contact
FR8 (Capture Logging)	UC-5.1 (Write Capture Record)	`capture_write_frame()`	**TC-4.2**	Written file opens in Wireshark; byte-exact vs. source
FR9 (Telemetry Stream)	UC-5.2 (Stream Telemetry)	`ws_broadcast_telemetry()`	**TC-5.1**	Dashboard refresh 500 to 1000 ms; fault alert ≤ 200 ms
NFR1 (Timing Resolution)	UC-1.1 (Hardware Timer)	`tk_ptmr_*()`	**TC-1.1**	Physical timer resolution ≤ 1 µs confirmed on logic analyzer
NFR2 (Voltage Compliance)	UC-1.1 (Direct 3.3V Logic)	Probe Driver (Direct 3.3V)	**TC-1.4**	Direct 3.3V bus levels (0.0 V to 3.3 V ± 5%); rise time degradation ≤ 50 ns
NFR5 (Zero Dynamic Heap)	UC-1.1 (Memory Allocation)	`tk_get_mpf()` / `tk_rel_mpf()`	**TC-4.3**	0 heap allocations over ≥ 10 min continuous run

NFR8 (Jitter < 500 ns)	`UC-3.1` (Glitch Precision)	`timer_sync_glitch()`	**TC-3.1**	Edge-to-pulse jitter < 500 ns on oscilloscope
NFR9 (Burst Reliability)	`UC-2.1` (Queue Reliability)	`ring_buffer_push()`	**TC-2.1**	≤ 0.1% drop at 500 kbps; ≥ 32-frame burst buffered